

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Technical Presentation

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I **Yarra Pawan (192424294)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Y. Pawan**

Technical Presentation Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Content	30%	Content is highly accurate, relevant, and insightful	Content is correct with minor gaps	Basic content with limited depth	Content is inaccurate or superficial
Structure and Flow	20%	Excellent logical flow and slide organization	Good structure with minor issues	Some organization but weak flow	Disorganized presentation
Use of Visuals	15%	Visuals strongly support technical communication	Relevant visuals used well	Limited or unclear visuals	No meaningful visuals
Communication Skills	20%	Confident, clear, and engaging delivery	Clear delivery with minor hesitation	Moderate clarity but uneven delivery	Weak delivery and low clarity

Professionalism	15%	Well-prepared and audience-focused presentation	Mostly professional	Basic preparation visible	Poor preparation and professionalism
------------------------	------------	--	----------------------------	----------------------------------	---

Total Marks Awarded:

Questions:

- 1) **Garbage In, Garbage Out: Why Data Cleaning Matters**
- 2) **From Messy Data to Meaningful Data**
- 3) **Detecting the Hidden Problems in Data**
- 4) **Finding the Unusual: A Study of Outliers**
- 5) **Are These Records Really Different? Understanding Duplicates**
- 6) **When Names Don't Match: The Role of Fuzzy Matching**
- 7) **Regex in Action: Cleaning and Validating Text Data**
- 8) **Making Data Consistent: Normalization and Standardization**
- 9) **Automating Data Cleaning with Python**
- 10) **Can a Cleaning Script Handle New Data?**
- 11) **Cleaning Real-World Financial Data**
- 12) **Cleaning Customer Data for Better Business Decisions**
- 13) **The Cost of Dirty Data in Organizations**
- 14) **Building a Reliable Data Cleaning Pipeline**
- 15) **From Raw Dataset to Analysis-Ready Dataset**

Signature of the Course Faculty



DSA05003 – QUERY PROCESSING

Regex in Action

Cleaning and Validating Text Data

Guided By

Dr C Anuradha

Yarra Pawan (192424294)

B TarunReddy (192424213)

M V Raghavan (192424373)

AGENDA

What We'll Cover

01

What Is Regex?

Pattern matching, the core idea

02

Why Clean Text?

The messy reality of raw data

03

Building Blocks

Anchors, classes, quantifiers, groups

04

Pattern Library

Email, phone, URL, whitespace, HTML

05

Case Study

A real cleaning pipeline, before/after

06

Live Validation

Testing input against a pattern

07

Advanced Extraction

Named groups for structured data

08

Ecosystem

Re, js, pandas.str

What Is a Regular Expression?

A regular expression (regex) is a **compact pattern language** for describing text — not a literal string to search for, but a *shape* that many different strings can match.

- Built into nearly every language: Python (re), JavaScript, Java, SQL, shell tools
- One engine reads left to right, trying to satisfy the pattern against the text
- Same syntax powers both validation (does it match?) and cleaning (replace what matches)

```
^[A-Za-z0-9_]+@[A-Za-z0-9]+\.[A-Za-z]{2,}$
```



^ \$ Anchors — start / end of string



[] Character class — any one of these



(2,) Quantifiers — how many repeats



\. Escape — treat . as a literal dot

Why Text Needs Cleaning

Raw text pulled from forms, scraped pages, and logs is rarely analysis-ready. Regex gives a precise, repeatable way to normalize it.



Raw Input

```
Name : " RAHUL  kumar"
Email: "Rahul.K@Mail"
Phone: "(91) 98765- 43210"
Note : "<b>VIP</b> customer!!"
Date : "07/08/26, 7-8-2026"
```



Cleaned Output

```
Name : "Rahul Kumar"
Email: "rahul.k@email.com"
Phone: "+91 9876543210"
Note : "VIP customer"
Date : "2026-08-07"
```

The Regex Toolkit



Anchors

Pin a match to the start or end of the string / line



Character Classes

Match any single character from a defined set, e.g. `[a-z0-9]`



Quantifiers

Control repetition — zero+, one+, optional, or `{n,m}` times



Groups

Capture a sub-match to reuse, extract, or reference later



Alternation

Match one option out of several, like `cat|dog|bird`



Shorthand Classes

Built-in shortcuts for digits, word chars, and whitespace

03

A Working Pattern Library

Use Case	Pattern	Matches
Email	<code>^[\w,+~]+@[\w-]+\.[a-zA-Z]{2,}\$</code>	<code>rahul.k@mail.com</code>
Indian phone	<code>^(?!(\+91[-])?[6-9]\d{9})\$</code>	<code>+91 9876543210</code>
URL	<code>https?://[\w.-]+\.[a-z]{2,}(/\S*)?</code>	<code>https://site.in/page</code>
Extra whitespace	<code>\s{2,}</code>	<code>collapse to a single space</code>
HTML tags	<code><[<^>]+></code>	<code>strip markup from scraped text</code>

04

Cleaning a Messy Customer Dataset

A single chained `re.sub()` pipeline normalizes names, collapses whitespace, and lower-cases emails — in one readable pass.
Syntax = `re.sub(pattern, replacement, string)`

```
import re

def clean_record(text: str) -> str:
    text = re.sub(r'\s+', ' ', text).strip() # collapse whitespace
    text = re.sub(r'<[^>]+>', '', text)      # strip HTML tags '<h>VIP</h> customer' -> VIP customer
    text = re.sub(r'([A-Za-z])\1{2,}', r'\1', text) # aaaa -> a
    return text.title()

record["email"] = record["email"].strip().lower()
record["name"] = clean_record(record["name"])
```

05

Validating Input in Real Time

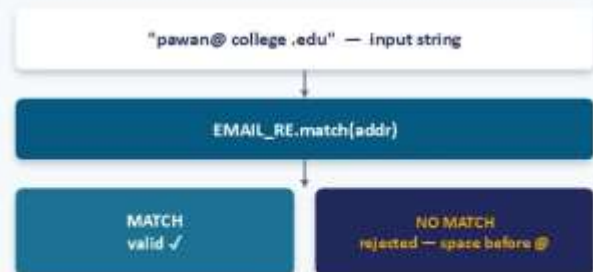
```
import re

EMAIL_RE = re.compile(
    r'^([\w.+-]+)@([\w-]+\.[a-zA-Z]{2,})$'
)

def is_valid_email(addr: str) -> bool:
    return bool(EMAIL_RE.match(addr))
```

```
is_valid_email("pawan@college.edu")
→ True
```

Trailing/inner spaces, missing @, or a bare TLD all fail the same check — the pattern is the single source of truth.



The engine tests the whole pattern against the string — one stray space and the anchors `^/$` reject it outright.

06

Structured Extraction with Named Groups

Named capture groups turn a flat log line into structured fields — no manual string-splitting required.

```
log = "2026-08-10 14:46:17 ERROR user=pawan action=login"
```

```
pattern = (
    r'(?P<date>\d{4}-\d{2}-\d{2}) (?P<time>\d{2}:\d{2}:\d{2}) '
    r'(?P<level>\w+) user=(?P<user>\w+) action=(?P<action>\w+)'
)
match = re.match(pattern, log)
match.groupdict()
```

```
{'date': '2026-08-10', 'time': '14:46:17', 'level': 'ERROR', 'user': 'pawan', 'action': 'login'}
```

09

ECOSYSTEM

Tools That Make Regex Practical



Python re module

compile, match, search, sub, findall



JavaScript RegExp

test(), match(), replace() in the browser



regex101.com

Live tester with a step-by-step explainer



pandas .str

Vectorized regex cleaning across a column



Editor find/replace

VS Code & most IDEs support regex search



Shell tools

grep, sed, awk for command-line text ops

11

Thank You

Piwan Tanunreddy & Raghavan · Regex in Action: Cleaning and Validating Text Data